

源码深度拆解



原来 Claude Code 的 Agent 框架 是这么设计的

1900个文件 · 51万行TypeScript
泄露源码里藏着哪些精妙设计？

多Agent协作

记忆系统

缓存命中

权限分层

自动压缩

Dream机制

~40

Agent 工具

~50

Slash 命令

7

核心子系统

基于 2026.03.31 泄露源码分析 | 逐文件阅读



rednote ID: 8995051376

精妙设计 ②：三层记忆系统

Agent 怎么"记住"你，又怎么"忘掉"不重要的

💡 核心洞察

Claude Code 有**三层独立的记忆机制**，各管各的事：

- ① 手动记忆 (MEMORY.md) → 用户主动写
- ② 自动记忆 (extractMemories) → 每轮对话结束后自动提取
- ③ Dream 记忆整合 → 后台定时合并、去重、精简

🧠 自动记忆提取 (extractMemories)

每次对话的 query loop 结束后 (模型输出了没有 tool call 的最终响应)，会自动 fork 一个子代理去分析本轮对话，提取值得长期保存的记忆。

关键：它用的是 `runForkedAgent` ——**共享父级 prompt cache**，所以这个"提取记忆"的操作几乎不额外花钱。

🔄 Dream：后台记忆整合

源码里有个 `autoDream` 服务，灵感很明显来自人类睡眠时的**记忆巩固**。

触发条件：距离上次整合 ≥ 24 小时 + 累积 ≥ 5 个会话。

做的事：回顾最近的会话记录，合并重复记忆，清理过时信息。

有专门的 `consolidationLock` 防止多进程同时跑 Dream。

UI 上会在底部状态栏显示 "dreaming..." 的任务条。

对话进行中 → 上下文持续累积

对话结束时 → `extractMemories` 提取关键信息

后台定时 → `autoDream` 合并 · 去重 · 精简

下次对话 → `MEMORY.md` + `auto memory` 注入 `system prompt`

这套设计让 Agent 真正拥有了"长期记忆"，而不是每次从零开始

rednote ID: 8995051376

精妙设计 ③: Swarm 多代理

不是一个 Agent 干所有事，而是一群 Agent 协作

💡 核心洞察

Claude Code 内部有一套完整的**"Swarm"多代理协作系统**。

一个 coordinator (指挥官) 可以同时 spawn 多个 teammate (队友)，每个队友在独立的 Git worktree 里工作，互不干扰，最后各自提 PR 合并。

📑 两种 Spawn 方式

进程隔离：通过 tmux/iTerm2 开新终端窗口，每个 teammate 是独立进程

进程内隔离：用 Node.js 的 `AsyncLocalStorage` 在同一个进程里跑多个 Agent，共享内存但上下文隔离。

进程内方式更轻量，启动更快，但进程隔离更安全。
源码里两种方案并存，通过 feature flag 控制。

📦 Batch 编排 (bundled skill)

内置了一个叫 **"batch"** 的 Skill，专门做大规模并行重构：

1. Coordinator 先进入 Plan Mode，研究代码库
2. 把任务拆成 5-30 个独立单元
3. 每个单元分配给一个 Worker Agent
4. Worker 在独立 worktree 里改代码 → 跑测试 → 提 PR
5. Coordinator 收集所有 PR 链接

这就是为什么 Claude Code 能一次重构上百个文件。

💬 SendMessageTool — Agent 间通信

Teammate 之间通过 `SendMessageTool` 互发消息。

coordinator 可以动态增减队友 (`TeamCreateTool` / `TeamDeleteTool`)。

权限通过 `leaderPermissionBridge` 上报给主 Agent 统一决策。

rednote

rednote ID: 8995051376

这套架构很像分布式系统的 leader-Worker 模式

精妙设计 ④：上下文自动压缩

对话太长怎么办？不是截断，是“浓缩”

💡 核心洞察

Claude Code 不会在上下文满了以后粗暴截断。它有一套**智能压缩系统**：当 token 数接近上限时，自动 fork 一个子代理来“总结”前面的对话，然后用总结替换掉原始消息，**同时保留 session memory**。

⚡ autoCompact 触发逻辑

每轮对话后计算当前 token 数，如果超过 **context window - 20000**（留给输出的空间），就触发自动压缩。

压缩失败有**熔断机制**：连续失败会停止重试，避免 context 已经不可恢复时（比如 prompt_too_long）陷入死循环。

🔄 压缩不只是“总结”

压缩前后还会执行 hooks：

- **preCompactHooks** — 压缩前保存重要状态
- **postCompactCleanup** — 压缩后清理缓存、重新注入记忆
- **sessionMemoryCompact** — 连 session 级记忆也一起压缩

同时，被读过的文件状态（fileStateCache）会被保留，这样压缩后 Agent 不需要重新读已经看过的文件。

🔧 microCompact — 更细粒度的压缩

除了整体压缩，还有一个 **microCompact**，可以只压缩**单个 tool result**（比如一个巨长的文件读取结果），而不是把整个对话都压缩掉。

还有基于时间的 **timeBasedMCConfig**，根据消息的“年龄”决定压缩力度——越老的消息，压得越狠。

这套设计让超长编码任务成为可能，而不会中途“失忆”。ednote ID: 8995051376

精妙设计 ⑤：权限 & Feature Flag

怎么让 Agent 既强大又安全、又能灰度发布

 **核心洞察**

Claude Code 的权限不是简单的"允许/拒绝"。它有**三种处理路径**：
本地 Hooks 快速判断 → Bash 分类器（AI 判断命令危险度）→ 人工审批弹窗。
只有前两层都搞不定，才会真正弹窗问你。

 **权限处理三级瀑布**

第一级：Permission Hooks
本地规则匹配，毫秒级。比如"读文件永远允许"、"在 node_modules 里写入永远拒绝"。

第二级：Bash Classifier
用 AI 模型分析 bash 命令的危险度。只对 Bash 工具生效，其他工具跳过。

第三级：Interactive Dialog
前两级都无法决策时，弹出权限对话框让用户选 allow/deny。Coordinator 模式下，子代理的权限会**上报到主 Agent** 统一处理。

 **Feature Flag 做死代码消除**

用 Bun 构建时的 `feature()` 函数做条件编译：

PROACTIVE — 主动模式（Agent 会自己干活）
KAIROS — 移动端/远程模式
COORDINATOR_MODE — 多代理指挥官模式
FORK_SUBAGENT — Fork 子代理
VOICE_MODE — 语音输入
DAEMON — 守护进程模式
TEAMMEM — 团队记忆同步

构建时不启用的 feature，**代码直接从产物中删除**，不是 if/else 运行时判断，是物理上不存在。

 **企业管控（MDM）**

rednote ID: 8995051376

总结：5个值得学的设计

从 Claude Code 源码里能偷到什么师

- 1 Fork 子代理共享 Prompt Cache
子 Agent 精确克隆父级的请求前缀，API 调用直接命中缓存，成本降一半以上
- 2 三层记忆 + Dream 整合
手动记忆 → 自动提取 → 后台定时合并去重。让 Agent 真正有了"长期记忆"
- 3 Swarm 多代理 Leader-Worker
Coordinator 拆任务，Worker 在独立 worktree 各自干活，最后各提 PR
- 4 智能上下文压缩（不是截断）
接近 token 上限时自动总结、微压缩、按消息年龄分级处理，保留文件状态缓存
- 5 权限三级瀑布 + 编译期 Feature Flag
Hooks → AI分类器 → 人工审批，层层过滤。feature() 做物理级死代码消除

 **我的看法**

Claude Code 的核心竞争力不只是模型好。

它真正厉害的是把**缓存优化、记忆系统、多代理协作、上下文管理**这些工程细节做到了极致。

这些设计单独拿出来都不复杂，但组合在一起，才构成了一个**真正能干活**的 Agent 框架。

预计下周国内就会出现一批"最接近 Claude Code"的 xxx code。
但能不能做到**这个工程深度**，是另一回事了。

- #ClaudeCode
- #Agent框架
- #源码分析
- #AI开发
- #大模型

How Claude Code Works

首页

从源码中发现的关键设计洞察

架构层

Agent 循环层

上下文工程层

安全层

工程细节层

系统架构

文档目录

快速入门

专题深入

关键数据

阅读建议

相关项目

贡献

License

10 分钟快速入门

核心架构

> 1. 概述

> 2. 系统主循环

> 3. 上下文工程

能力系统

> 4. 工具系统

> 5. 代码编辑策略

> 9. Hooks 与可扩展性

> 10. 多 Agent 架构

重试，后台任务（摘要、预测）直接放弃。防止后台任务在级联故障时放大网关压力。

系统架构

```
graph TD; UserInput[用户输入] --> QEM[QueryEngine 会话管理]; UserInput --> CE[上下文工程]; QEM --> QMC[query 主循环]; QMC --> CA[Claude API 调用]; CA --> PR{解析响应}; PR -- 文本 --> FO[流式输出]; PR -- 工具调用 --> TIE[工具执行引擎]; CE --> ST[系统提示词]; CE --> GS[Git 状态]; CE --> CL[CLAUDE.md]; CE --> CLF[压缩流水线]; TIE -- 结果回注 --> QMC; TIE --> RS[权限系统]; RS --> RL[规则层]; RS --> BAST[Bash AST 分析]; RS --> UC[用户确认]; TIE --> RW[读文件]; TIE --> WF[编辑文件]; TIE --> SE[Shell 执行]; TIE --> ST[搜索工具]; TIE --> MCP[MCP 工具];
```

文档目录

快速入门

• 10 分钟读懂 Claude Code — 全部内容的浓缩版，适合快速了解

专题深入

rednote ID: 2169512777

第二幕：Harness 框架高调发布（3 月 24 日）

Anthropic 工程团队发布了一篇长达数千字的技术博客，系统阐述 Harness Design 理念。

- | 模型能力已经不再是瓶颈，工程编排才是
- | 提出了 GAN 式多智能体架构（生成器 + 评估器 + 规划器）
- | 展示了用框架实现 6 小时、\$200 成本产出全栈应用的案例
- | 隐含结论：Claude Code 的价值不在于模型本身，而在于这套工程体系

这一篇才是真正暴露底牌的文章。Anthropic 自己在说：模型竞赛的天花板已经到了，接下来拼的是工程和编排能力。但这也意味着——当他们将框架方法论公开的时候，Claude Code 的核心壁垒正在被自己拆解。

第三幕：Claude Code 全部源码泄露（3 月 31 日）

REDACTED

rednote ID: 969747605

安全研究员 Chaofan Shou 发现 npm 包中的 Source Map 文件未移除，导致 51.2 万行源代码完全暴露。

- | 1900 个 TypeScript 文件
- | 约 40 种工具的完整实现
- | 查询引擎（46000 行），处理所有 LLM API 调用和任务编排
- | 多智能体编排系统、IDE 桥接系统、持久化记忆系统
- | 约 50 个斜杠命令
- | 未发布功能：Buddy AI 宠物、KAIROS 持久化助手模式

51.2 万行代码，1900 个文件。如果你是一个想要"不小心"泄露源代码的公司，你很难做得比这更彻底了。

| 为什么说全是套路

000075

rednote ID: 969747606

逻辑链一：泄露 = 事实上的开源

让我们说句大实话：当你的全部源代码已经在 GitHub 上被镜像了几百次的时候，"闭源"这个标签还有什么实际意义吗？没有。

Claude Code 的代码现在对全世界开发者来说已经是公开的了。竞品可以逐行研究架构设计，开源社区可以自由 fork。唯一的区别只是法律层面——Anthropic 仍然持有版权，理论上可以追责。但在 AI 编码工具这个领域，架构思想和工程模式的复制几乎不受版权约束。

所以，泄露之后，Claude Code 在技术层面已经等同于开源了。Anthropic 面临的选择只有两个：

1. 继续假装闭源，显得小气且无力
2. 顺势正式开源，赢得一波"拥抱开放"的好名声

在 IPO 前，你觉得他们会选哪个？

逻辑链二：Claude Code 的天花板已经到了

CLI 编码工具这个赛道的游戏规则正在发生变化。

Claude Code 面临的天花板：

REDACTED

rednote ID: 969747606

- | 本质上是一个 CLI 工具，架构已基本定型，迭代空间有限
- | 源码泄露意味着技术壁垒归零——任何团队都可以逐行复刻
- | 模型能力的边际收益在快速递减——Anthropic 自己在 Harness 文章里都承认了这一点

但整个 agentic coding 生态的天花板却还在快速升高：

- | 各种 CLI 工具如雨后春笋——Aider、Cline、OpenHands，还有没死透的 Cursor、Windsurf……
- | Codex + OpenClaw 这类组合拳，把能力拼到了新高度
- | OpenClaw + 开源模型开源界的路线正在证明，不需要顶级闭源模型也能构建足够强的编码 Agent

结论：整个赛道的天花板还在升高，但是Claude

REDS

rednote ID: 969747606

Code 看到了自己的天花板。

此时开源 Claude Code，是把它从"触及天花板的闭源产品"变成"行业基础设施"，让它在生态的下一个上升周期里继续占位——完成一次战略意义上的资产重估。

逻辑链三：开源模型正在吞噬 Anthropic 的护城河

2026 年的开源格局已经不再是 2024 年了，尤其是中国军团：

- | DeepSeek 4：马上发布（希望尽快），综合性能绝对直逼顶级闭源模型，成本只有几分之一
- | Qwen 3：开源扛把子，多语言能力极强，Apache 2.0 协议完全自由
- | Kimi（Moonshot）：k2.5 事实上的行业标准，超长上下文王者，200 万 token 不是噱头
- | GLM（智谱）：企业级深耕，中文代码理解一流

REDA

rednote ID: 969747606

- | Mimo (小米): 又一头黑马之姿冲出来的, 绝对会开源
- | MiniMax: 多模态和语音交互突出, 产品化速度快

企业客户的算账逻辑越来越简单: 当开源模型已经"够好"的时候, 为什么要为闭源模型支付 10 倍的价格? 这些中国模型在核心场景上已经从"追赶"变成了"够用", 甚至"好用"。

Anthropic 的"宪法 AI"、安全合规、幻觉率 <1% 确实仍是差异化优势。但这种优势在缩小, 缩小速度在加快。Claude Code 如果开源, 反而能建立"Anthropic 生态"——即使底层模型可以被替换, 工程框架和工具链可以绑定开发者。

逻辑链四: IPO 叙事需要"开放"标签

\$3800 亿估值, 27 倍收入倍数, \$600 亿 IPO。这种规模的发行, 投资者需要的是一个长期故事, 不是一个季度数据。

"我们是一个封闭的 API 供应商"——这个故事天花板很低。

REDS

rednote ID: 969747606